

[← Go Back](#)

Hardware

Nano RP2040 Connect

Tutorials

[Nano RP2040 Connect User Manual](#)[Web Server AP Mode with Arduino Nano RP2040 Connect](#)[BLE Device to Device with Nano RP2040 Connect](#)[Nano RP2040 Connect Chromebook Setup](#)[Nano RP2040 Datalogger with MicroPython](#)[Using the IMU Machine Learning Core Features](#)[Accessing IMU Data on Nano RP2040 Connect](#)[Setting up Nano RP2040 Connect with Arduino Cloud](#)[Reading Microphone Data on Nano RP2040 Connect](#)[MicroPython and the IMU Machine Learning Core Features](#)[Getting Started With the Nano RP2040 Connect and OpenMV](#)[Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect](#)[Voice Commands With The Arduino Speech Recognition Engine](#)

Home / Hardware / Nano RP2040 Connect / [Reading Microphone Data on Nano RP2040 Connect](#)



This tutorial refers to a product that has reached its end-of-life status.

Reading Microphone Data on Nano RP2040 Connect

Learn how to read data from the MP34DT06JTR microphone, and how to use the data to turn ON or OFF the built-in RGB.

Author · Karl Söderby · Last revision · 11/13/2024

Introduction

The Nano RP2040 connect comes with the **MP34DT06JTR** microphone, which can be used to record audio. In this tutorial, we will setup a basic application that simply turns ON or OFF the built in RGB LED whenever a loud noise is recorded (for example snapping our fingers).

Note: if you need help setting up your environment to use your Arduino Nano RP2040 board, please refer to [this installation guide](#).

ON THIS PAGE

Introduction

[Goals](#)[Hardware & Software Needed](#)[The MP34DT06JTR Microphone](#)[Programming the Board](#)[Testing It Out](#)[Conclusion](#)[Help](#)

The goals of this project are:

- Learn about PDM.

- Read microphone data.

- Print out the data in the Serial Monitor / Serial Plotter.

- Create trigger value(s).

- Activate an RGB based on a loud noise.

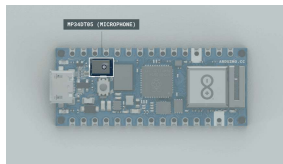
Hardware & Software Needed

- Arduino IDE ([online](#) or [offline](#)).

- [PDM](#) library installed.

- [Arduino Nano RP2040 Connect](#).

The MP34DT06JTR Microphone



The MP34DT06JTR microphone.

Microphones are components that convert physical sound into digital data. Microphones are commonly used in mobile terminals, speech recognition systems or even gaming and virtual reality input devices.

The MP34DT06JTR sensor is a ultra compact microphone that

Modulation) to represent an analog signal with a binary signal. The sensor's range of different values are the following:

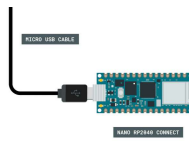
Signal-to-noise ratio: 64dB

Sensitivity: -26dBFS $\pm$ 3dB

Temperature range: -40 to 85°C

Circuit

This tutorial requires no additional circuit. You will only need to connect the board to a computer through a Micro USB cable.



The circuit.

Programming the Board

We will now get to the programming part of this tutorial.

- 1 First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board**

Arduino Mbed OS

Nano Boards and

install it.

- 2 We can now take a look at some of the core functions of this sketch:

```
static const char  
channels = 1;
```

- sets the number of output channels.

```
static const int  
frequency = 20000;
```

- sets the sampling frequency to 20 KHz.

```
short sampleBuffer[512]
```

- buffer to read samples into, each sample is 16-bits.

```
while (!Serial) -
```

prevents program from running until Serial Monitor is opened.

```
PDM.begin(channels,  
frequency)
```

- initializes the PDM library.

```
Serial.print(sampleBuff  
er[i])
```

- prints sample to the Serial Monitor / Plotter.

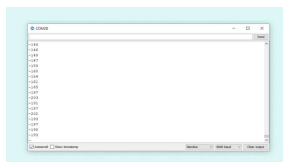
The sketch can be found in the snippet below. Upload the sketch to the board.



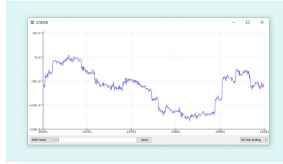
```
1 #include <WiFiNINA.h>
2 #include <PDM.h>
3
4 bool LED_SWITCH = false;
5
6 // default number of
7 static const char channel = 'A';
8
9 // default PCM output
10 static const int frequency = 44100;
11
12 // Buffer to read samples
13 short sampleBuffer[512];
14
15 // Number of audio samples read
16 volatile int samplesRead = 0;
17
18 void setup() {
19     Serial.begin(9600);
20     pinMode(LED_BUILTIN, OUTPUT);
21     while (!Serial);
22     // Configure the PDM
23     PDM.onReceive(onPDMReceive);
24
25     // Optionally set the gain
26     // Defaults to 20 dB
27     // PDM.setGain(30);
28 }
```

Testing It Out

After successfully uploading the code to the board, we will need to open the Serial Monitor to initialize the program. Once we open it, we can see that data is being printed rapidly. These are the audio samples that are recorded from the microphone. We can also open the Serial Plotter to instead view the data as a graph.



We can also open the Serial Plotter to instead view the data as a graph.



Serial Plotter with microphone data.

Now in the sketch, we have written a conditional that is activated whenever the sample value is either over `10000` or under `-10000`. This can be produced, by for example snapping our fingers, coughing loudly or if you prefer, yelling (we prefer the finger snapping method).

Whenever this condition is met, the following things happen:

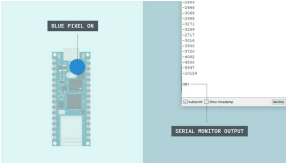
The boolean `LED_SWITCH` changes.

Depending on the state of `LED_SWITCH`, the blue pixel turns ON or OFF.

The text `"ON"` or `"OFF"` is printed in the monitor.

The program freezes for one second, then resumes (this is just to allow some time to see the status of the LED).

In the Serial Monitor we should see the following when the conditional is met, where the **blue** pixel will turn ON / OFF depending on the state of `LED_SWITCH`.



The blue RGB pixel
either turns ON/OFF
when the condition is
met.

Conclusion

In this tutorial we have learned how to retrieve data from the microphone onboard the Nano RP2040 Connect board. The sketch we used is an elaboration of the **PDM** example that can be found through **File > Examples > PDM > PDMSerialPlotter**.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

